

МЕТОДИЧЕСКИЕ УКАЗАНИЯ**УП.03. Учебная практика****Этап 4. Тестирование и диагностика качества****Анализ рисков и характеристик качества ПО**

Параметр	Значение
Практика	УП.03. Сопровождение и обслуживание программного обеспечения компьютерных систем
Этап 4	Тестирование и диагностика качества программной системы после локального запуска и развертывания
Связь с этапом 2	На этапе 2 проект был подготовлен к запуску через BAT/Makefile/Docker, настроены линтеры и форматтеры
Связь с этапом 3	На этапе 3 проект был развернут как сервис, demo-стенд, PaaS-развертывание или готовый билд
Основной результат	Качество проекта проверено не глазами, а инструментами: тесты, DevTools, Lighthouse, Postman/Swagger/curl, pytest, k6/JMeter, анализ логов
Формат сдачи	Ссылка на репозиторий + короткий отчет + тест-план + журнал дефектов + отчет о рисках + скриншоты/отчеты инструментов
Инструменты	Chrome DevTools, Lighthouse, Swagger, Postman, curl, pytest, JMeter/k6, logs, браузерная консоль, Network/Performance

Главная мысль этапа:

Студент не должен писать “всё работает”. Он должен доказать это: запустить проверки, показать логи, протестировать сценарии, найти дефекты, оценить риски и сделать вывод о качестве проекта.

Что должно получиться после этапа

После выполнения этапа у проекта должен появиться минимальный комплект контроля качества. Проверяются не только внешний вид и запуск, но и основные сценарии, API, ошибки в консоли, производительность, логи, устойчивость после перезапуска и риски эксплуатации.

Что должно быть	Зачем это нужно	Минимальный результат
TEST_PLAN.md	Понять, что именно проверялось	Есть список 8-12 проверок: UI/API/БД/ошибки/логи/производительность
DEFECT_LOG.md	Не скрывать ошибки, а фиксировать их нормально	Минимум 3 найденные проблемы или обоснование, почему критических проблем нет
RISK_REGISTER.md	Связать качество с рисками эксплуатации	Минимум 5 рисков с вероятностью, влиянием и способом снижения
QUALITY_REPORT.docx	Коротко зафиксировать итог этапа	2-4 страницы: что проверено, какими инструментами, вывод
Postman/Swagger/curl-проверки	Доказать работу API или основных функций	3-5 успешных запросов + 1 ошибочный сценарий
Lighthouse/DevTools	Проверить web-интерфейс не только визуально	Отчет или скриншоты Network/Console/Performance/Lighthouse
Автотесты или smoke-тесты	Повторяемая проверка после изменений	pytest/npm test/gradle test или checklist для desktop/console
Логи	Понять, есть ли ошибки после запуска	Скриншот/файл с логами без критических ошибок

Логика перехода от этапа 3 к этапу 4

Было на этапе 3	Что делаем на этапе 4
Проект развернут как сервис, demo-стенд или готовый билд	Проверяем, что развернутый результат реально работает и не ломается в основных сценариях
Есть production/demo-конфигурация	Проверяем переменные окружения, подключение к БД, доступность адресов, CORS/порты/режим запуска
Есть deploy/restart/check_deploy-скрипты	Проверяем поведение после перезапуска, смотрим логи и доступность
Есть открываемый интерфейс или API	Тестируем UI, API, основные маршруты, ошибки в консоли и сетевые запросы

Было на этапе 3	Что делаем на этапе 4
Есть release/demo-сборка	Проверяем, можно ли ее открыть, пройти основной сценарий и повторить запуск

Важно:

Этап 4 не повторяет деплой. Если проект не запускается или не развернут, сначала исправить этап 2/3. На этапе 4 проверяется качество уже подготовленного результата.

Структура сдачи этапа 4

Папка сдачи

```

УП03_Этап4_Иванов_Иван_Группа/
|-- repo_link.txt
|-- docs/
|   |-- TEST_PLAN.md
|   |-- DEFECT_LOG.md
|   |-- RISK_REGISTER.md
|   |-- QUALITY_REPORT.docx
|   |-- PERFORMANCE_REPORT.md
|   `-- API_TEST_REPORT.md
|-- reports/
|   |-- lighthouse_report.html или lighthouse_summary.png
|   |-- postman_collection.json
|   |-- postman_run_result.json или screenshots
|   |-- pytest_report.txt / npm_test_report.txt / gradle_test_report.txt
|   |-- k6_summary.txt или jmeter_report.png
|   `-- logs_tail.txt
|-- screenshots/
|   |-- 01_deployed_app_opened.png
|   |-- 02_devtools_console_no_critical_errors.png
|   |-- 03_network_requests_success.png
|   |-- 04_lighthouse_or_performance.png
|   |-- 05_api_success_request.png
|   |-- 06_api_error_request_handled.png
|   |-- 07_tests_success.png
|   |-- 08_logs_without_critical_errors.png
|   |-- 09_defect_before_after.png
|   `-- 10_git_commit.png
|-- scripts/
|   |-- test.bat
|   |-- api-test.bat
|   |-- quality-check.bat
|   |-- performance.bat
|   `-- logs-check.bat
`-- tests/
    |-- smoke/
    |-- api/
    `-- load/ # если используется k6/JMeter
  
```

Порядок выполнения

Шаг	Что сделать	Что приложить
1	Открыть развернутый проект из этапа 3 или demo/release-сборку.	01_deployed_app_opened.png
2	Составить TEST_PLAN.md: основные сценарии, API, ошибки, логи, производительность.	Скриншот/файл TEST_PLAN.md
3	Проверить UI через DevTools: Console, Network, Performance.	02_devtools_console_no_critical_errors.png, 03_network_requests_success.png
4	Проверить Lighthouse или аналогичную диагностику для web-проекта.	04_lighthouse_or_performance.png или lighthouse_report.html
5	Проверить API через Swagger/Postman/curl. Выполнить успешные и ошибочные запросы.	05_api_success_request.png, 06_api_error_request_handled.png
6	Запустить тесты или smoke-проверки одной командой.	07_tests_success.png + отчет tests
7	Проверить логи приложения после основных сценариев.	08_logs_without_critical_errors.png + logs_tail.txt
8	Зафиксировать дефекты в DEFECT_LOG.md и исправить хотя бы часть проблем.	09_defect_before_after.png
9	Составить RISK_REGISTER.md по качеству и эксплуатации.	Таблица рисков
10	Сделать commit/push всех файлов и отчетов.	10_git_commit.png

Обязательные команды этапа 4

Действие	BAT-команда	Makefile-команда	Что должно быть видно
Запуск тестов	scripts\test.bat	make test	Тесты прошли или показали конкретные ошибки
Проверка API	scripts\api-test.bat	make api-test	Postman/curl/pytest проверили 3-5 запросов
Общая проверка качества	scripts\quality-check.bat	make quality-check	Одна команда запускает ключевые проверки
Проверка производительности	scripts\performance.bat	make performance	Lighthouse/k6/JMeter или ручной Performance report
Проверка логов	scripts\logs-check.bat	make logs-check	Логи открыты, критических ошибок нет или они описаны

Пример quality-check.bat

```
# scripts/quality-check.bat
@echo off
chcp 65001 > nul
cd /d "%~dp0\.."

echo =====
echo Общая проверка качества проекта
echo =====

call scripts\test.bat
call scripts\api-test.bat
call scripts\logs-check.bat

echo Проверка завершена. Сохраните скриншот результата.
```

Фрагмент Makefile

```
# Makefile: фрагмент этапа 4
.PHONY: test api-test quality-check performance logs-check

test:
    # пример: pytest -q или npm test или gradlew test

api-test:
    # пример: newman run reports/postman_collection.json или curl-команды

quality-check: test api-test logs-check
    @echo "Quality check completed"

performance:
    # пример: npx lighthouse http://localhost:5173 --output html --output-path reports/lighthouse_report.html
    # пример: k6 run tests/load/basic_load.js

logs-check:
    # пример: docker compose logs --tail=100 > reports/logs_tail.txt
```

Что проверять по типу проекта

Тип проекта	Минимум проверки	Инструменты
Web/frontend	Страница открывается; нет критических ошибок в Console; основные запросы Network успешны; Lighthouse/Performance выполнен; проверена адаптивность	Chrome DevTools, Lighthouse, Network, Console, Performance
Backend/API	Swagger открывается; 3-5 успешных запросов; 1 ошибочный запрос корректно обработан; есть коды 200/201/400/401/404; логи без 500-ошибок	Swagger, Postman, curl, pytest, logs
Fullstack	Проверить оба контура: UI + API + БД. Действие в интерфейсе должно создавать/изменять данные в backend/БД	DevTools, Postman, БД, logs, pytest
Desktop/game	Запуск билда; основной сценарий; обработка ошибки; проверка настроек; проверка логов/консоли	Скриншоты, логи, checklist, unit/smoke tests
Console/CLI	Запуск с корректными данными; запуск с ошибочными данными; вывод результата; обработка исключений; smoke-тест	BAT, pytest, screenshots, logs

Шаблон TEST_PLAN.md

```
# TEST_PLAN.md

## 1. Что проверяется
Проект: <название>
Версия / ветка: <branch / commit>
Адрес / способ запуска: <URL или команда>

## 2. Основные сценарии
| ID | Сценарий | Ожидаемый результат | Инструмент | Статус |
|----|-----|-----|-----|-----|
| TC-01 | Открыть главную страницу | Страница открылась без ошибок | Browser/DevTools | passed |
| TC-02 | Выполнить вход | Пользователь вошел в систему | UI/API | passed |
| TC-03 | Создать запись/заявку/объект | Объект создан и отображается | UI + API | passed |
| TC-04 | Ошибочный ввод | Пользователь видит понятную ошибку | UI/API | failed/passed |
| TC-05 | Проверить логи | Нет критических 500/Traceback | logs | passed |

## 3. Инструменты
- DevTools Console/Network/Performance:
- Lighthouse:
- Postman/Swagger/curl:
- pytest/npm test/gradle test:
- k6/JMeter, если использовалось:

## 4. Итог
Критичные ошибки: <количество>
Некритичные ошибки: <количество>
Вывод: <готов/не готов к дальнейшей эксплуатации>
```

API-проверки: Swagger, Postman, curl

Если у проекта есть API, студент обязан проверить не только успешный запрос, но и ошибочный сценарий. Например: запрос без токена, несуществующий ID, пустое обязательное поле, неверный логин/пароль.

Проверка	Что выполнить	Что приложить
Успешный GET	Получить список объектов или один объект	Скриншот Postman/Swagger/curl с кодом 200
Успешный POST/PUT	Создать или изменить запись	Скриншот кода 201/200 и тела ответа
Ошибочный запрос	Отправить неверные данные или запрос без доступа	Скриншот кода 400/401/404 и понятной ошибки
Проверка БД	Показать, что действие изменило данные	Скриншот таблицы/админки/ответа API
Проверка логов	Посмотреть логи после запросов	logs_tail.txt или скриншот логов

Пример curl-команд

```
# Пример curl-проверок
curl -X GET http://localhost:8000/api/health
curl -X GET http://localhost:8000/api/items
curl -X POST http://localhost:8000/api/items -H "Content-Type: application/json" -d '{"title":"demo"}'
curl -X GET http://localhost:8000/api/items/999999
```

Проверка web-интерфейса через DevTools

Раздел DevTools	Что проверить	Не принимается
Console	Нет красных критических ошибок при открытии и основном сценарии	Скриншот только страницы без консоли
Network	Основные запросы имеют коды 200/201/304, ошибки 400/401/404 объяснены	Не видно URL/статусов запросов
Performance	Показан замер или краткий вывод о скорости	Нет никакого замера
Lighthouse	Есть отчет или скриншот основных метрик	Просто написано “проверил Lighthouse”
Responsive	Проверен хотя бы один мобильный размер	Интерфейс проверен только на одном размере

Производительность: минимальная проверка

Не нужно делать промышленное нагрузочное тестирование. Нужно показать, что студент умеет измерять, а не оценивать “на глаз”.

Вариант	Что сделать	Результат
Web	Запустить Lighthouse или DevTools Performance	Скриншот/HTML-отчет + 2-3 вывода

Вариант	Что сделать	Результат
API	Выполнить серию запросов через Postman/curl/pytest/k6	Среднее время ответа или скриншот результатов
Backend + БД	Проверить 1-2 тяжелых операции: список, поиск, создание записи	Время ответа, логи, вывод
Desktop/console	Замерить время выполнения основного сценария	Скриншот/описание результата

Пример простого k6-теста

```
// tests/load/basic_load.js для k6, если проект поддерживает HTTP API
import http from 'k6/http';
import { check } from 'k6';

export default function () {
  const res = http.get('http://localhost:8000/api/health');
  check(res, {
    'status is 200': (r) => r.status === 200,
    'response time < 1000ms': (r) => r.timings.duration < 1000,
  });
}
```

Журнал дефектов DEFECT_LOG.md

Дефект — это не обязательно “всё сломалось”. Это любая найденная проблема: ошибка в консоли, неверный статус API, медленный запрос, неправильная валидация, непонятное сообщение, отсутствие лога.

# DEFECT_LOG.md						
ID	Где найдено	Описание проблемы	Как воспроизвести	Критичность	Статус	Исправление
BUG-01	UI / форма входа	При пустом пароле нет понятного сообщения	Открыть вход, нажать Войти	Средняя	fixed	Добавлена валидация
BUG-02	API / items	Несуществующий ID возвращает 500 вместо 404	GET /api/items/999999	Высокая	fixed	Добавлена обработка not found
BUG-03	Logs	Ошибка не записывалась в лог	Вызвать ошибочный запрос	Средняя	open	Запланировано логирование

Реестр рисков RISK_REGISTER.md

Риск — это проблема, которая может проявиться при эксплуатации. На этом этапе важно связать результаты проверки с рисками качества.

Риск	Признак/доказательство	Вероятность	Влияние	Что сделать
Проект недоступен после перезапуска	После restart сервис не поднялся	Средняя	Высокое	Добавить restart policy, проверить env и логи
Ошибки API приводят к 500	Ошибочный запрос без обработки	Высокая	Высокое	Добавить обработчики исключений и тесты
Медленная загрузка страницы	Lighthouse/Performance показывает задержки	Средняя	Среднее	Оптимизировать ресурсы, запросы, сборку
Пользователь видит непонятные ошибки	UI показывает технический traceback/stack	Средняя	Среднее	Добавить пользовательские сообщения
Логи не помогают в диагностике	Нет записи ошибки или контекста	Средняя	Высокое	Настроить уровни логирования и формат сообщений

Короткий отчет QUALITY_REPORT.docx

Раздел отчета	Что написать
1. Ссылка на проект	Репозиторий, ветка, commit, адрес развернутого проекта или путь к build/release
2. Что проверялось	UI, API, БД, логи, производительность, перезапуск, основной сценарий
3. Инструменты	DevTools, Lighthouse, Postman/Swagger/curl, pytest/npm test, k6/JMeter, logs
4. Результаты	Сколько проверок пройдено, какие ошибки найдены, что исправлено
5. Дефекты	Краткий список BUG-01, BUG-02, BUG-03 и их статус
6. Риски	Краткий список главных рисков и меры снижения

Раздел отчета	Что написать
7. Вывод	Готов ли проект к дальнейшей эксплуатации, что нужно улучшить дальше

Обязательные скриншоты

Файл	Что должно быть видно
01_deployed_app_opened.png	Развернутый проект открыт в браузере/окне/терминале
02_devtools_console_no_critical_errors.png	Console открыта, критических красных ошибок нет или они описаны
03_network_requests_success.png	Network: видны URL, статусы, время запросов
04_lighthouse_or_performance.png	Lighthouse/Performance/аналогичный замер
05_api_success_request.png	Успешный API-запрос: код 200/201 и тело ответа
06_api_error_request_handled.png	Ошибочный запрос: код 400/401/404, а не непонятный 500
07_tests_success.png	Команда тестов/smoke-check выполнена
08_logs_without_critical_errors.png	Логи после проверки, критических ошибок нет или они занесены в дефекты
09_defect_before_after.png	Пример найденной и исправленной проблемы
10_git_commit.png	Commit/push с файлами этапа 4

Критерии приемки этапа

Критерий	Принимается	Не принимается
Проверка инструментами	Есть реальные отчеты/скриншоты DevTools, Lighthouse/Postman/pytest/logs	Студент просто пишет “всё работает”
Тест-план	Есть TEST_PLAN.md с конкретными сценариями и статусами	Тест-план состоит из общих фраз
API/UI	Проверены успешные и ошибочные сценарии	Проверен только один “красивый” экран
Дефекты	Ошибки занесены в DEFECT_LOG.md, часть исправлена или обоснована	Ошибки скрыты, журнал пустой без объяснения
Риски	Есть минимум 5 рисков с мерами снижения	Риски не связаны с проектом
Скриншоты	Скриншоты доказывают команды, запросы, логи и результаты	Скриншоты не показывают проверку
Итоговый вывод	Есть честный вывод о качестве и готовности к эксплуатации	Написано “проект полностью готов” без доказательств

Минимум для зачета:

Проект открыт, есть тест-план, проверены UI/API или основной сценарий, есть логи, есть дефекты/риски, есть скриншоты и commit. Без доказательств инструментами этап не засчитывается.